GO DATABASE ENGINE + PYTHON TORTURE FRAMEWORK

Complete system architecture, execution flow, storage logic, recovery design, and performance measurements

Project goal
Store a dataset of approximately 1,000 people in a binary database with CRC32 integrity checks. Build the database engine in Go. Use Python as an independent test and torture controller to interrupt the Go process, restart it, validate recovered data, and measure retrieval and restoration time.

Core principle: Go is responsible for database correctness. Python tries to break the engine and independently checks whether its guarantees hold.

Architecture specification • Implementation guide • Test strategy

1. The flow from your notebook

The notebook shows this intended sequence: a dataset of 1,000 people is stored in binary form with CRC; the stored database is tortured; a pass/fail decision is made; then retrieval time and restoration time are measured, followed by the total time needed to restore the 1,000-person dataset.

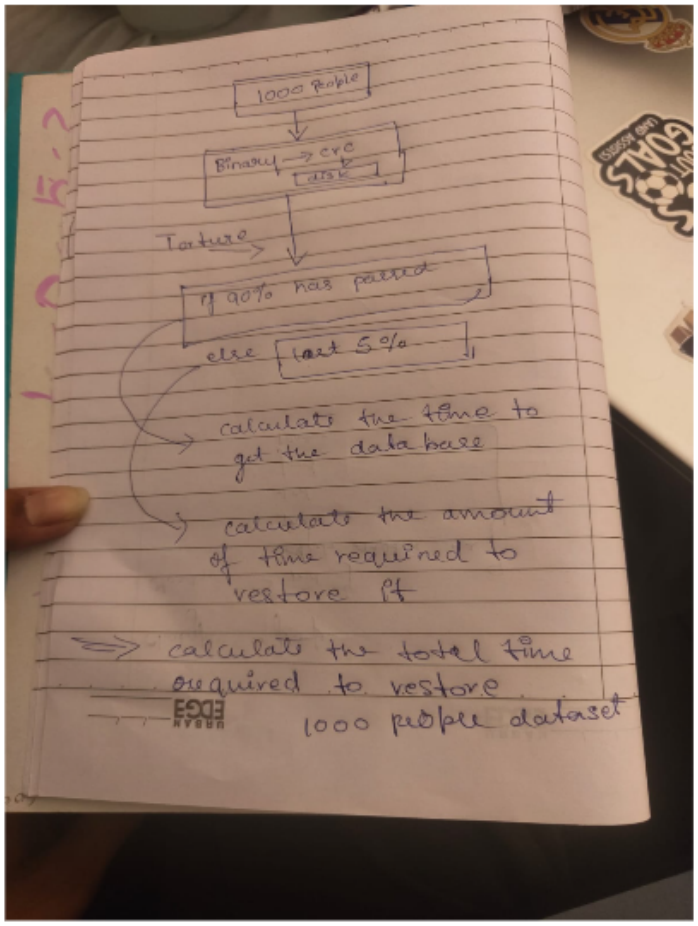


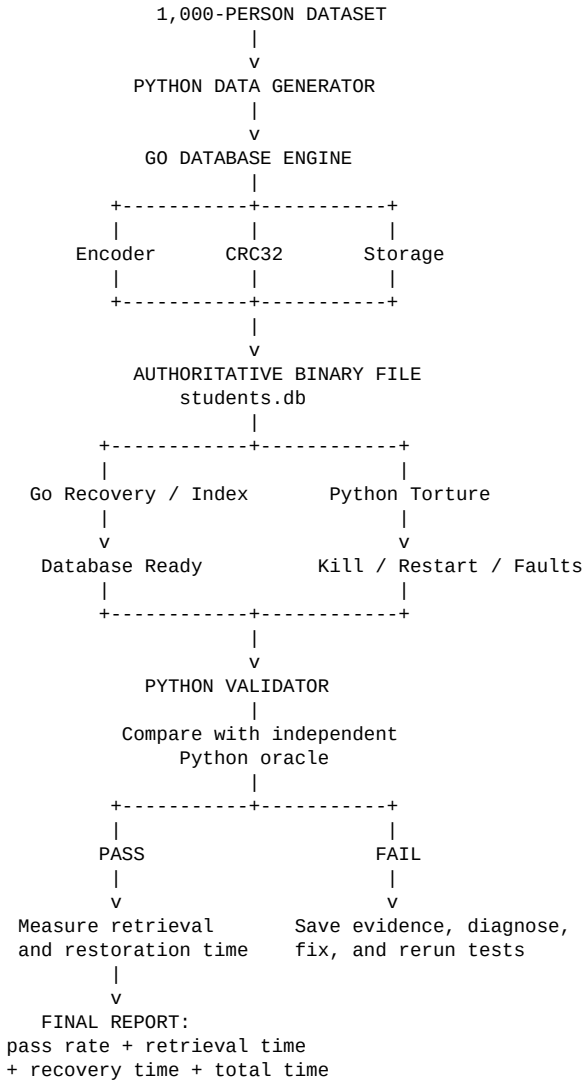
Figure 1. Uploaded notebook flow (cropped for readability).

Important interpretation note: the handwritten decision appears to say “if 90% has passed” and then “else last 5%”. Those percentages do not define a complete mathematical rule as written (90% pass and a 5% remainder leave 5% unexplained). This document preserves the idea as a configurable acceptance gate, but the project owner must confirm whether the intended threshold is 90%, 95%, or a separate final-5% failure/retry stage. Do not hard-code this ambiguity.

Operational interpretation used below

1) Build and verify the 1,000-person database. 2) Run normal, stress, and crash-torture tests. 3) Calculate the percentage of test cases that pass. 4) If the agreed acceptance threshold is met, run timed retrieval and recovery measurements. 5) If it is not met, preserve the failing test, diagnose the fault, fix the engine, and rerun. 6) Report retrieval time, recovery time, and total restoration time separately.

2. Overall system architecture



Responsibility boundary

Go engine	Python framework
Binary format, CRC, writes, reads, deletes, batch transactions, index, recovery, compaction, concurrency control.	Dataset generation, command orchestration, independent oracle, process termination, restart, fault injection, validation, timing, reporting.
Owns the database file and decides which operations are committed.	Does not decide database truth from Go's own output; it compares the recovered state against its own expected state.

Communication

Python starts Go in a long-running server mode and communicates using JSON Lines over stdin/stdout. Standard output is reserved for protocol responses; diagnostic logs go to stderr. Python must be able to detect a dead process, restart it, and reconnect.

```
Python test runner
    |  JSON request line
    v
Go process (serve mode)
    |  JSON response line / ACK
    v
Python test runner

Example request:
{"op":"PUT","id":"S1001","value":{"name":"Asha","age":20}}
```

```
Example successful response:
{"status":"ACK","operation_id":"op-000001"}
```

3. Go engine: modules and responsibilities

Package	Responsibility and logic
cmd/database	Program entry point; parses flags such as serve, check, dump, and benchmark.
engine	Coordinates operations and enforces their order: validate → encode → durable append → index update → ACK.
records	Defines the student record and validation rules for IDs, names, age, department, and email.
binary	Encodes and decodes a versioned, explicit binary format; never writes raw Go struct memory.
crc	Calculates and verifies CRC32 over the defined frame bytes.
storage	Owns file opening, append, read-at-offset, sync, file-size checks, and safe close behavior.
indexing	Maintains an in-memory map from StudentID to the latest committed record location.
transaction	Validates and encodes a batch as one atomic transaction frame.
recovery	Scans valid frames from the beginning, rebuilds the index, and handles an incomplete final tail.
compaction	Writes a compact snapshot to a temporary file, validates and syncs it, then safely replaces the old file.
protocol	Parses JSON-line commands, returns structured responses, and separates protocol output from logs.

Suggested repository layout

```
student-database/
  cmd/database/main.go
  internal/
    engine/
    records/
    binary/
    crc/
    storage/
    indexing/
    transaction/
    recovery/
    compaction/
    protocol/
  database/students.db
  python_tests/
  test_data/
  reports/
  go.mod
```

4. Binary file format and CRC logic

Use one authoritative append-only file, for example students.db. The file is the source of truth. The in-memory index is disposable and rebuilt after restart. This avoids the additional consistency problem of keeping a separate persistent index file synchronized with the data file.

Recommended frame layout

```
+----------------------+  fixed magic bytes
| MAGIC                |
+----------------------+
| FORMAT_VERSION       |
+----------------------+
| FRAME_TYPE           |  PUT / DELETE / BATCH / SNAPSHOT
+----------------------+
| SEQUENCE_NUMBER      |
+----------------------+
| PAYLOAD_LENGTH       |
+----------------------+
| PAYLOAD              |  encoded operation or transaction
+----------------------+
| CRC32                |  checksum over specified frame fields
+----------------------+
| FOOTER / END_MARKER  |
+----------------------+
```

The decoder must impose maximum frame and payload sizes before allocating memory. It must reject invalid magic, unsupported versions, impossible lengths, invalid frame types, checksum mismatch, and invalid footer markers.

PUT write path

1. Validate student record and key.
2. Encode operation into bytes.
3. Build complete frame and calculate CRC32.
4. Append the frame to students.db.
5. Call file.Sync() (fsync-equivalent request).
6. Update in-memory index.
7. Return ACK only after the durability step succeeds.

GET path

1. Acquire read lock.
2. Look up StudentID in the in-memory index.
3. Read the referenced frame from the file.
4. Validate frame boundaries and CRC32.
5. Decode the value and return it.

CRC is an integrity check, not a durability protocol

CRC32 detects accidental corruption with high probability. It does not make a write durable, does not make a batch atomic, and is not protection against deliberate tampering. Durability depends on the write protocol and successful sync; atomicity depends on transaction framing and recovery rules.

5. Record operations, index, and transaction semantics

Append-only versioning

An update appends a new PUT for the same StudentID rather than overwriting the earlier frame. The newest committed operation wins. A DELETE appends a tombstone. The recovery scanner applies operations in sequence order, so the index ends with the latest live value for each key.

```
File history:
PUT S1001 {age:20}
PUT S1002 {age:21}
PUT S1001 {age:22}   <- newest value for S1001
DELETE S1002         <- S1002 is no longer live

Rebuilt index:
S1001 -> offset of latest PUT
S1002 -> absent
```

Index design

Start with a Go map[string]RecordLocation. It provides expected constant-time key lookup on average. The map is only an acceleration structure: losing it must not lose data. Recovery rebuilds it from the file. Sequential scanning remains useful for validation and benchmarks.

Atomic batch design

For the first reliable implementation, encode an entire batch as one framed transaction. Validate every operation before writing. Append the single complete batch frame, sync it, then apply all its changes to the in-memory index under an exclusive lock. Readers must not observe a partially applied batch.

```
BATCH transaction frame:
  sequence = 104
  payload = [
    PUT S1001 ...,
    PUT S1002 ...,
    DELETE S1003
  ]
  CRC32 = checksum of complete frame

Recovery:
  valid complete frame   -> apply every operation
  invalid/incomplete tail -> apply none of this frame
```

Concurrency plan

Begin with one serialized writer and concurrent readers protected by an RWMutex. This is intentionally conservative. Add multiple writers only after single-writer durability, batch atomicity, and recovery are proven. Use Go's race detector as an additional development check.

6. Durability and crash recovery

The ACK is the durability boundary. Go must not acknowledge a write until the complete transaction frame has been appended and the required sync call has succeeded.

Crash timing	Permitted result after restart
Before any frame bytes are written	Operation absent.
During frame append	Incomplete tail rejected; operation absent.
After append, before successful sync	Operation may or may not survive; it was not ACKed.
After sync, before ACK reaches Python	Operation may be present. No ACK does not mean absent.
After ACK	Operation must be present after successful recovery.

Recovery algorithm

```
1. Open the authoritative database file.
2. Validate the file header and supported format version.
3. Start scanning at the first frame.
4. Read and validate each frame's length and structure.
5. Verify CRC32 and footer.
6. Apply valid operations in sequence order to a fresh in-memory index.
7. If the final frame is incomplete, truncate to the last valid frame boundary.
8. If corruption is found in the middle, stop and report corruption;
   do not silently skip it.
9. Mark the database ready only after recovery finishes.
```

Crash during recovery

For the initial implementation, rebuild the in-memory index from the authoritative file on every startup. Do not persist partial recovery progress. If recovery itself is killed, its in-memory state disappears; the next startup repeats the scan from the beginning. This makes recovery restartable and deterministic, although it may require optimization to meet a strict time target.

Filesystem caveat

For crash-safe replacement and compaction, the implementation must account for file sync, atomic rename behavior on the target filesystem, and syncing the parent directory after rename where required. Validate these assumptions on the operating systems and filesystems used for evaluation.

7. Compaction and file-size management

Append-only files grow because updates leave older versions and deletes leave tombstones. Compaction creates a new file containing only the current live state.

```
1. Acquire exclusive writer lock.
2. Read current live records from the index.
3. Create a temporary file in the same filesystem/directory.
4. Write a valid snapshot with sequence metadata and checksums.
5. Validate the temporary file by reading it back.
6. Sync the temporary file.
7. Atomically rename it over the active database file.
8. Sync the parent directory where required.
9. Reopen/update file handles and release the lock.
```

Crash expectations

Crash point	Expected safe state
Before temporary snapshot is complete	Old database remains authoritative; temporary file can be discarded.
After temp sync, before rename	Old database remains authoritative; complete temp file is not yet active.
During/after atomic rename	After restart, exactly one complete authoritative version should be selected according to filesystem rename semantics.
After rename, before directory sync	Power-loss guarantees may depend on filesystem; test and document platform assumptions.

File-size limit

Make the maximum database size configurable. Before appending, check whether the operation would exceed the configured limit. Return a clear error without ACK if it cannot be safely committed. Test limits at boundaries and during compaction; do not assume free-space checks eliminate races or guarantee a successful write.

8. Python test and torture framework

Python is the controller and independent verifier. It should communicate with Go through the supported protocol for ordinary operations. Direct file manipulation is reserved for controlled tests on disposable copies, such as truncation and selected corruption scenarios.

```
python_tests/
  main.py
  config/settings.py
  generator/dataset.py
  client/go_client.py
  oracle/expected_state.py
  functional/test_crud.py
  stress/test_performance.py
  torture/
    crash_test.py
    batch_test.py
    recovery_test.py
    concurrency_test.py
    disk_full_test.py
    truncation_test.py
    compaction_test.py
  validator/verify_database.py
  reports/report.py
```

Python modules

Module	Logic
dataset.py	Creates deterministic 1,000-person dataset; supports larger sizes and varied value lengths.
go_client.py	Starts Go, sends JSON-line requests, reads responses, detects process death, restarts, and captures logs.
expected_state.py	Maintains independent expected state from acknowledged operations and records sent/ACKed operation IDs.
functional tests	Checks PUT, GET, UPDATE, DELETE, duplicate keys, invalid inputs, and batch behavior.
torture tests	Kills Go at random times and at controlled failpoints; tests recovery, concurrency, disk full, truncation, and compaction.
validator.py	Obtains a deterministic read-only dump/check result and compares it with oracle expectations and allowed uncertain operations.
report.py	Calculates pass rate, retrieval latency, recovery latency, total restore time, and failure evidence.

9. Oracle rules and torture execution

> The Python oracle is the independent ground truth for what must be durable. It must never infer expected state from the database's own dump.

For every request, Python records the operation ID, request contents, send time, response, ACK status, test seed, and crash phase. Once Go returns ACK, Python applies that operation to its durable expected-state log. On restart, every ACKed operation must be reflected in the database.

Unacknowledged operations need special handling: they may have reached durable storage before the process died but the ACK was not delivered. The test oracle must allow the outcomes permitted by the transaction protocol. For a batch, the whole batch may be present or absent, but never partially applied.

Random kills and deterministic failpoints

Random SIGKILL-style termination explores unpredictable timing. It is not sufficient alone: a test may run hundreds of random kills without hitting a critical instruction boundary. Add test-only failpoints in Go and have Python terminate the process at each target boundary.

```
Suggested failpoints:
  AFTER_FRAME_WRITE
  BEFORE_FSYNC
  AFTER_FSYNC
  BEFORE_INDEX_UPDATE
  DURING_RECOVERY
  DURING_COMPACTION

Torture loop:
  start Go
  send operations and record ACKs
  trigger random or targeted crash
  terminate Go abruptly
  restart Go
  wait for recovery-ready response
  query deterministic state
  compare with Python oracle
  save seed, operation log, crash point, and result
  repeat
```

Do not enable failpoints in ordinary production builds. The test build should make them controllable through an explicit environment variable or test-only control command.

10. Test matrix: what must be proved

Test	Python action	Pass condition
CRUD	Generate and execute valid/invalid record operations.	Returned values and final state match expected semantics.
Single-write crash	Kill at append, pre-sync, post-sync, and pre-ACK points.	All ACKed writes survive; incomplete frame is not returned as valid.
Batch crash	Kill while a multi-operation batch is being written.	Batch is entirely applied or entirely absent.
Recovery crash	Kill during index rebuild; restart again.	Recovery completes deterministically and can be repeated.
Concurrency	Run multiple readers/writers and batches.	No corruption, lost ACKed writes, or partial batch visibility.
Disk full	Constrain storage or inject write errors.	No false ACK; prior committed state remains recoverable.
Truncation	Truncate disposable copies at multiple tail offsets.	Recover to last valid frame; report middle-file corruption.
Compaction crash	Kill at each compaction stage.	Old or new complete database survives; no mixed state.
Repeated abuse	Repeat randomized and targeted kill/restart cycles.	All invariants continue to hold; failures are reproducible.

Five project invariants

Invariant	Meaning
Durability	Every acknowledged operation is present after recovery.
Atomicity	A batch is fully applied or fully absent.
No torn values	Incomplete or checksum-invalid frames are never returned as valid records.
Recovery	The database reopens after crashes, including a crash during recovery.
Concurrency	Concurrent access does not corrupt state or violate transaction guarantees.

11. Exact measurement flow from your diagram

The notebook's final section asks for three timing results. Measure them independently; do not combine them into one number without showing the components.

A. Time to get the database (retrieval time)

Define a repeatable query workload, for example: retrieve all 1,000 StudentIDs once, or retrieve a fixed random sample. Record the start immediately before the query workload and stop after all expected results have been validated. Report the workload, number of queries, and latency distribution.

```
retrieval_time = end_time_of_validated_queries - start_time_of_queries

Report:
  total queries
  successful queries
  failed queries
  total elapsed time
  average latency
  p50 / p95 / p99 latency
```

B. Time required to restore it (recovery time)

This is the time from launching the Go process against the post-crash database until Go reports that recovery is complete and the database is ready for requests. Do not include the Python dataset-generation time.

```
recovery_time = database_ready_timestamp - process_start_timestamp

Record:
  database file size
  number of valid frames scanned
  number of live keys rebuilt
  number of invalid tail bytes discarded
  recovery errors, if any
```

C. Total time to restore the 1,000-person dataset

If the database file is already present after a crash, the restoration time is normally the recovery time. If your intended experiment includes regenerating and reloading all 1,000 people, measure that as a separate full-rebuild scenario. Do not add retrieval time to recovery time unless the definition of 'restore' explicitly requires retrieving and validating every record.

```
Define two separate metrics:

Crash recovery:
  T_crash_restore = T_process_start_to_ready

Full dataset rebuild (only if required):
  T_full_restore =
      T_generate_or_load_input
    + T_write_and_sync_1000_records
    + T_recovery_or_reopen
    + T_validate_all_1000_records

Optional end-to-end validation:
  T_end_to_end =
      T_crash_restore
    + T_retrieve_and_validate_all_1000_records
```

D. Pass percentage gate

```
pass_rate_percent = (passed_test_cases / total_test_cases) * 100

Acceptance:
  pass_rate_percent >= configured_threshold

Do not assume the handwritten 90% / last 5% notation is final.
Confirm the intended threshold before using it as a release gate.
For core durability invariants, a single confirmed violation should fail
the release regardless of the aggregate percentage.
```

12. Performance plan and acceptance criteria

Start with the requested 1,000-person dataset. Then increase the scale to reveal performance and recovery bottlenecks. A one-million-key recovery target, if required by the challenge brief, must be benchmarked on documented hardware and storage; it is not guaranteed by the architecture alone.

Dataset size	Purpose	Measurements
1,000	Baseline and functional validation.	Load, retrieve-all, crash recovery, full validation.
10,000	Small-scale growth.	Read/write latency, file size, memory, recovery.
100,000	Medium-scale performance.	Throughput, p95/p99 latency, compaction, recovery.
1,000,000	Large-scale target if required.	Recovery duration, memory, file scan rate, indexed lookup.

Suggested measurable release gates

Area	Acceptance rule
Durability	Zero acknowledged-operation losses in the torture suite.
Batch atomicity	Zero partially applied batches.
Recovery	Every supported crash scenario reopens or reports a clear unrecoverable corruption error; no silent data skipping.
Retrieval	All requested IDs return correct values; publish latency measurements rather than an unsupported speed claim.
Restore	Publish measured recovery time for 1,000 records and, if required, one million keys.
Reproducibility	Every failure report includes seed, operation history, crash point, binary version, OS, and database copy.

13. Recommended implementation order

Phase	Build	Language	Exit condition
1	Student schema and validation	Go	Valid and invalid records are handled correctly.
2	Binary frame encoder/decoder and CRC32	Go	Round-trip and malformed-frame tests pass.
3	Append-only storage and sync	Go	Frames append and can be read back.
4	PUT / GET / DELETE and in-memory index	Go	CRUD works without restart.
5	Batch frame and atomic index application	Go	Batch is all-or-nothing in recovery tests.
6	Recovery scanner and tail handling	Go	Index rebuild is deterministic and restartable.
7	JSON-line serve protocol	Go	Python can command and restart Go.
8	Python generator, client, oracle	Python	Oracle independently tracks ACKed operations.
9	Functional tests and failpoints	Both	Targeted crash windows are testable.
10	Random torture, concurrency, disk-full, truncation	Python + Go hooks	Five invariants are automatically checked.
11	Compaction and compaction torture	Go + Python	Old/new complete state survives injected crashes.
12	Timing, scale tests, report	Python	Retrieval, recovery, and total restore metrics reported separately.

Do not begin with concurrent writers or advanced compaction. First prove a correct single-writer engine, durable ACK protocol, and repeatable recovery. Then add complexity.

14. Final deliverables and definition of done

The project is complete when the following can be demonstrated from a clean checkout:

Deliverable	Evidence
Go database engine	Binary storage, CRC32, CRUD, batch, index, recovery, compaction, configurable file-size limit.
Python test framework	Dataset generator, Go client, independent oracle, functional and torture tests.
Crash evidence	Random and targeted crash runs with reproducible seeds and logs.
Integrity evidence	CRC/frame validation results and explicit handling of tail vs middle-file corruption.
Timing report	Retrieval time, recovery time, and clearly defined total restoration time.
Scale report	Results for 1,000 records and larger datasets, with hardware and filesystem details.

Final engineering cautions

• CRC32 is not a substitute for fsync, transaction framing, or recovery.
• An unacknowledged operation may still be present after restart; the oracle must model this correctly.
• Random process kills do not equal power-loss testing. Be precise about what failure model is being tested.
• A process kill tests application crash behavior, not every storage-device or filesystem power-loss behavior.
• Never run destructive truncation or disk-full tests against the only copy of important data.
• An aggregate 90% pass rate must not excuse a durability or atomicity violation. Core invariants should have zero tolerated failures.

Recommended final result

A Go database engine that stores the 1,000-person dataset in a versioned binary file with CRC32 checks; a Python framework that generates workloads, independently tracks acknowledged operations, kills and restarts Go at random and deterministic points, verifies the recovered state, and reports retrieval and restoration timings.

End of architecture specification